```
# Experiment 1
# Titanic Dataset Preprocessing

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Load Dataset
df = pd.read_csv(r"C:/Users/digit/Downloads/Titanic-Dataset.csv")

# Display first 5 rows
print("First 5 Rows:")
print(df.head())

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Fill missing values
df['Age'].fillna(df['Age'].mean(), inplace=True)

df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Drop Cabin column
df.drop(['Cabin'], axis=1, inplace=True)

# Convert categorical data into numerical
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Remove duplicate rows
df.drop_duplicates(inplace=True)

# Feature Scaling
scaler = StandardScaler()
df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

# Final Dataset
print("\nPreprocessed Dataset:")
print(df.head())

print("\nDataset Info:")
print(df.info())



??????????????????????????????????????????

# Experiment 2
# Sentiment Analysis using KNN

import pandas as pd

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Create Dataset
data = {
    'text': [
        'good movie',
        'excellent film',
        'amazing acting',
        'bad movie',
```

```
        'worst film',
        'poor acting'
    ],

    'sentiment': [
        'positive',
        'positive',
        'positive',
        'negative',
        'negative',
        'negative'
    ]
}

# Convert into DataFrame
df = pd.DataFrame(data)

print("Dataset:")
print(df)

# Convert text into numerical vectors
vectorizer = CountVectorizer()

X = vectorizer.fit_transform(df['text'])

y = df['sentiment']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Apply KNN
model = KNeighborsClassifier(n_neighbors=3)

# Train Model
model.fit(X_train, y_train)

# Prediction
prediction = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, prediction)

print("\nPrediction:")
print(prediction)

print("\nAccuracy:")
print(accuracy)

??????????????????????????????????????????????????????????????


# Experiment 1
# Activation Functions in ANN

import numpy as np

# Input values
x = np.array([-2, -1, 0, 1, 2])

# Sigmoid Function
```

```
sigmoid = 1 / (1 + np.exp(-x))

# Tanh Function
tanh = np.tanh(x)

# ReLU Function
relu = np.maximum(0, x)

print("Input Values:")
print(x)

print("\nSigmoid Output:")
print(sigmoid)

print("\nTanh Output:")
print(tanh)

print("\nReLU Output:")
print(relu)

??????????????????????????????

# Experiment 2
# XOR using Back Propagation Neural Network

import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# XOR Input
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

# XOR Output
y = np.array([
    [0],
    [1],
    [1],
    [0]
])

# Create Model
model = Sequential()

# Hidden Layer
model.add(Dense(4, input_dim=2, activation='relu'))

# Output Layer
model.add(Dense(1, activation='sigmoid'))

# Compile Model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train Model
model.fit(X, y, epochs=500, verbose=0)
```

```
# Prediction
predictions = model.predict(X)

print("Predictions:")
print(predictions)

??????????????????????????????????????????

# Experiment 3
# Feed Forward Neural Network

import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Input Data
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

# Output Data
y = np.array([
    [0],
    [1],
    [1],
    [0]
])

# Create Model
model = Sequential()

# Hidden Layer
model.add(Dense(4, input_dim=2, activation='relu'))

# Output Layer
model.add(Dense(1, activation='sigmoid'))

# Compile Model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train Model
model.fit(X, y, epochs=500, verbose=0)

# Evaluate Model
loss, accuracy = model.evaluate(X, y, verbose=0)

print("Accuracy:", accuracy)

# Predictions
print(model.predict(X))
```